INFORME TP 3: CONJUNTO DOMINANTE MÍNIMO

Programación III - Universidad Nacional de General Sarmiento

2026

- **Docentes:** Patricia Bagnes - Daniel Bertaccini - Gonzalo Faccini
- **Alumnos:** Mendoza Tomás - Martínez Felipe - Villalba Javier
- **Comisión:** 01 | **Grupo:** 10



Introducción

El presente informe describe el análisis, diseño e implementación de una solución en Java para el **Problema del Conjunto Dominante Mínimo**, correspondiente al Trabajo Práctico 3 de la materia Programación III. El objetivo consiste en encontrar, dado un grafo no dirigido $G = (V, E)$, un subconjunto de vértices $A \subseteq V$ de tamaño tan pequeño como sea posible, tal que todo vértice que no pertenezca a A sea vecino de al menos un vértice en A.

Dado que la versión de decisión de este problema es de clase **NP-completo**, encontrar el óptimo global mediante algoritmos exactos requiere un costo computacional exponencial. Para resolver el problema en tiempos razonables de ejecución, se implementó una **heurística golosa.**

El desarrollo se realizó aplicando los principios fundamentales de la Programación Orientada a Objetos (POO), priorizando la alta cohesión, el bajo acoplamiento y el encapsulamiento de datos. Para estructurar la interacción entre la interfaz de usuario y la lógica de negocio, se adoptó el patrón de diseño **MVP (Model-View-Presenter)** en su variante de **Passive View (Vista Pasiva)**.

Presentación y Funcionamiento del Problema

Un conjunto dominante en un grafo es una cobertura de vértices donde cada nodo del grafo está seleccionado o bien es vecino de un nodo seleccionado.

1. Funcionamiento del Algoritmo Goloso

El algoritmo toma en cada paso una decisión localmente óptima basada en una heurística de máxima cobertura dinámica:

Estado Inicial: Se comienza con el conjunto dominante $A = \emptyset$ y el conjunto de vértices dominados $D = \emptyset$.
Iteración Golosa: Mientras $|D| < |V|$ (es decir, el conjunto de los NO dominados sea menor a los vértices totales):

- Se evalúan todos los vértices del grafo que aún no han sido incluidos en A.
- Se calcula la ganancia de cada vértice, definida como la cantidad de vértices vecinos que no están dominados
- Se selecciona el vértice (v^*) que maximice dicha ganancia y se incorpora a A.
- Se actualiza D agregando a v^* y a todos sus vecinos en el conjunto de dominados.

Resultado: Al finalizar el ciclo, A conforma un conjunto dominante válido y de tamaño reducido.

2. Modos de Carga del Grafo e Interacción

La aplicación Swing (cuyo tamaño visual respeta el límite máximo) permite la carga del grafo de dos maneras:

- **Carga Manual:** El usuario especifica la cantidad total de vértices V y agrega aristas una a una mediante sus vértices origen y destino.

- **Carga por Archivo (JSON):** Se permite seleccionar un archivo de texto plano con formato JSON. El sistema valida la integridad del archivo y construye la estructura del grafo automáticamente.
- **Visualización:** El sistema renderiza el grafo mediante la clase PanelGrafo, distribuyendo los nodos circularmente y resaltando en **rojo** los vértices pertenecientes al conjunto dominante y en **azul** los vértices restantes.

Arquitectura e Implementación

El proyecto está organizado en los paquetes modelo, vista, presentador, main y test.

1) Patrón de Diseño: Model-View-Presenter (MVP) - Passive View

Para dar cumplimiento al principio de *Separated Presentation*, la vista se diseñó de forma completamente pasiva: desconoce la existencia del modelo y no contiene código de respuesta a eventos ni lógica de decisión. MainPresenter actúa como mediador exclusivo.

Clase	**Clases Principales**	**Responsabilidad**
Modelo	Grafo, AlgoritmoGoloso, ResultadoDominante, CargadorGrafo	Encapsula la estructura matemática del grafo, ejecuta la heurística golosa y realiza la lectura/validación del archivo JSON.
Vista	MainFrame, PanelGrafo	Construye la interfaz Java Swing y renderiza gráficamente los vértices y aristas. No toma decisiones de negocio.
Presentador	MainPresenter	Escucha los eventos de Swing, valida la entrada de datos, invoca las clases de la capa Modelo y empuja las actualizaciones hacia la Vista.
Main	Main	Inicializa la vista y el presentador en el hilo de eventos de Swing

2) Modelo de Dominio y Responsabilidad de las Clases

- **Grafo**: representa un grafo no dirigido mediante una lista de conjuntos de vecinos. Valida la cantidad inicial de vértices, controla que los identificadores estén dentro del rango y rechaza los bucles. Permite agregar, eliminar y consultar aristas, obtener vecinos y consultar las cantidades de vértices y aristas.
- **AlgoritmoGoloso**: contiene el procedimiento estático resolver(Grafo). Mantiene los vértices dominados y el conjunto seleccionado. En cada iteración evalúa los vértices aún no elegidos y selecciona el que cubre más vértices nuevos.

- **ResultadoDominante**: encapsula el conjunto elegido. Copia el conjunto recibido y lo expone como no modificable, evitando que quien consulta el resultado altere su contenido. También ofrece un método para obtener el tamaño del conjunto.
- **CargadorGrafo**: lee un archivo UTF-8, interpreta su contenido JSON y valida su estructura y los datos antes de construir el objeto Grafo. Su analizador JSON está implementado dentro de la propia clase mediante la clase interna JsonParser.

3) Representación del grafo y complejidad

La clase Grafo utiliza una lista de vecinos: List<Set<Integer>>. La lista contiene una entrada por vértice y cada conjunto almacena sus vecinos. Como el grafo es no dirigido, al agregar una arista (i, j) se incorpora j en la lista de vecinos de i y también i en la de j. Los conjuntos evitan duplicar una misma conexión.

Operación	**Complejidad esperada**	**Justificación**
Crear Grafo	O(V)	Se crea una entrada de vecinos por cada vértice.
Agregar / eliminar / consultar arista	O(1) esperado	Las operaciones sobre HashSet son O(1); se actualizan o consultan dos conjuntos.
Obtener vecinos	O(1)	Se devuelve la vista no modificable del conjunto de adyacencia.
Espacio del Grafo	O(V + E)	Se almacena una entrada por vértice y dos referencias de adyacencia por arista no dirigida.

Complejidad del Algoritmo Goloso: En cada iteración, calcular los nuevos dominados de un vértice *v* toma $O(grado(v))$. Sumando sobre todos los vértices, cada paso toma $O(|V| + |E|)$. Como en el peor caso se realizan hasta $|V|$ iteraciones, la complejidad temporal global del algoritmo es $O(|V| \cdot (|V| + |E|))$.

4) Estructura del archivo JSON

Para la carga mediante archivos se definió la siguiente estructura en texto plano:

```
{
  "cantidadVertices": 5,
  "aristas": [
    [0, 1],
    [0, 2],
    [1, 3],
    [3, 4]
  ]
}
```

5) **Tests Unitarios**

Para garantizar la calidad de software requerida por la cátedra, la capa de negocio dispone de suites de pruebas automáticas desarrolladas en JUnit 4:

ClaseTest	**Cantidad de Test**	**Casos Cubiertos**
GrafoTest	8	Creación, agregado de aristas no dirigidas, prevención de duplicados, eliminación, cantidad de aristas, cantidad de vértices, rechazo de bucles y validación de identificadores. También comprueba que la colección de vecinos no pueda modificarse externamente.
AlgoritmoGolosoTest	3	Grafo estrella, grafo sin aristas y rechazo de grafo nulo. En los casos válidos se verifica que el resultado sea dominante.
CargadorGrafoTest	30	Carga correcta, orden variable de campos, espacios y saltos de línea, grafo sin aristas y rechazo de archivos vacíos o inválidos, campos faltantes o desconocidos, números incorrectos, aristas inválidas, bucles, duplicados y ruta nula.

Las pruebas del algoritmo verifican dos escenarios representativos, pero no realizan una comparación sistemática contra una solución óptima ni cubren grafos generales con múltiples empates o estructuras variadas. Por lo tanto, los tests comprueban propiedades y casos puntuales, pero no demuestran optimalidad del método goloso.

Problemas Encontrados y Soluciones Implementadas

1. **calculo Dinámico de Cobertura en la Heurística:**

   Problema: Si la heurística contaba únicamente el grado estático inicial de cada vértice, se seleccionaban nodos donde la cobertura se superponían con nodos ya dominados, produciendo conjuntos dominantes de mayor tamaño.

Solución se diseñó el método contarNuevosDominados(), el cual cuenta únicamente aquellos nodos vecinos que aún no pertenecen al conjunto de dominados D, de esta manera siempre se elije el vértice con mayor ganancia de vecinos.

2. **Ruptura del encapsulamiento en la Vista Pasiva (MVP)**
   Problema: Al pasar la lista de vecinos a componentes visuales como PanelGrafo, existía el riesgo de que la interfaz gráfica modificara directamente los conjuntos de vecinos de forma no controlada.
   Solución: En el método grafo.getVecinos(v) se retornó la colección protegida mediante Collections.unmodifiableSet(vecinos.get(v)). De esta forma, la vista pasiva puede consultar la estructura para dibujarla, pero no puede alterar el estado del modelo.

3. **Lectura de archivos JSON:**
   Problema: Durante la carga por archivo, la presencia de espacios en blanco, saltos de línea o texto adicional después del objeto JSON provocaba errores de formato al momento de construir el grafo.

   Solución: Se implementó un parser por recorrido de caracteres (JsonParser) dentro de CargadorGrafo que omite espacios en blanco, valida los corchetes de los arreglos y corrobora que las claves obligatorias (cantidadVertices y aristas) estén presentes antes de instanciar el grafo.